# Beyond Autocomplete: A Survey on the Evolution of Human-AI Pair Programming

Nikhil Anand
University of Massachusetts, Amherst
nikhilanand@umass.edu

## Abstract

Generative AI tools such as GitHub Copilot and ChatGPT are increasingly integrated into software development workflows, yet their influence on developers' problem solving strategies remains insufficiently understood. This paper presents a literature survey of ten recent studies examining how generative AI affects developer behavior during programming tasks. We adopt a hybrid analytical approach that first defines a taxonomy of AI-assisted interactions and then analyzes their impact across key stages of problem-solving, including problem understanding, solution generation, implementation, and debugging. Our synthesis shows that generative AI shifts developers from exploration-driven problem-solving toward solution selection and refinement, reducing cognitive effort while introducing risks of over-reliance and shallow reasoning. The impact varies across stages: conversational tools primarily support problem understanding, while suggestion-based and generative tools accelerate implementation and debugging. We further identify cross-cutting patterns in trust, prompting behavior, and learning. Finally, we highlight gaps in existing work, including limited longitudinal evidence and insufficient analysis of expertise-dependent effects. These findings provide a structured perspective on how generative AI reshapes developer problem-solving and suggest directions for future research.

***CCS Concepts*** • **Software and its engineering** → **Software development process management**; • **Computing methodologies** → *Multi-agent systems*; Information extraction; Natural language generation

## 1. Introduction

This project proposes an empirical study of AI-assisted programming beyond autocomplete, focusing on how generative AI tools influence developer workflows, decision making, and problem-solving strategies. Rather than evaluating the correctness or performance of generated code alone, the project investigates how developers interact with AI suggestions during real programming tasks and how these interactions shape development practices. The scope of the project includes examining developer behavior when using AI tools for tasks such as code generation, debugging, refactoring, and exploration of unfamiliar APIs. By analyzing these interactions, the project aims to understand whether AI assistants function primarily as productivity tools, collaborative partners, or exploratory aids during software development.

### 1.1 Scope and Goals

Ultimately, the goal of the project is to characterize the behavioral and workflow-level impacts of generative AI in programming environments. Through empirical observation and analysis of developer interactions with AI tools, the study seeks to identify patterns in how developers rely on AI assistance, the types of tasks where AI support is most beneficial, and the potential changes these tools introduce into software development practices. These insights can help inform the design of future developer tools and contribute to a broader understanding of human–AI collaboration in software engineering [2, 5].

The paper will look to tackle the following research questions[1]:

**RQ (1)** How do developers integrate generative AI tools into their programming workflows beyond simple code completion?

**RQ (2)** What types of programming tasks do developers most frequently use generative AI assistants for (e.g., code generation, debugging, refactoring, documentation)?

**RQ (3)** How does the use of generative AI tools influence developers' problem-solving strategies during programming tasks?

[1] These research questions are preliminary and may be refined based on the literature review and empirical findings.

**RQ (4)** To what extent do developers rely on AI-generated code compared to manually written code during development?

**RQ (5)** What challenges, limitations, or trust issues do developers encounter when collaborating with generative AI programming assistants?

### 1.2 Learning Outcomes

Through this project, the study aims to develop a clearer understanding of how human-AI collaboration influences developer productivity, cognition, trust, and code quality. By synthesizing existing empirical research, the work seeks to identify key patterns, limitations, and open challenges associated with AI-assisted programming. The findings are expected to contribute to a better understanding of how human-centered software engineering principles may need to evolve in the context of AI-supported development and AI-based pair programming.

## 2. Motivation

The rapid adoption of generative AI assistants in programming environments has raised important questions about how these tools influence software development practices. While early studies suggest that AI-powered tools can improve developer productivity and assist with a variety of programming tasks, their broader impact on developer behavior, decision making, and collaboration with automated systems remains insufficiently understood [1, 4]. Understanding these effects is essential for designing development tools that effectively support programmers while maintaining code quality, developer trust, and sustainable engineering practices.

Beyond productivity, AI-assisted programming may also reshape fundamental concepts within software engineering such as creativity, code comprehension, and development workflows. As developers increasingly collaborate with AI systems during programming tasks, traditional notions of authorship, problem-solving, and knowledge acquisition may evolve. Prior work on AI-supported programming and intelligent development tools suggests that these technologies can influence how developers explore solutions, reason about code, and learn new programming concepts, raising important questions about the future of human-centered software development [3, 5].

## 3. Planned Methodology

The project will be conducted as a structured literature review of empirical studies examining the use of generative AI tools in software development. Relevant papers will be identified through academic digital libraries such as Google Scholar, ACM Digital Library, IEEE Xplore, and arXiv using search terms related to AI-assisted programming, code generation, developer productivity, and human-AI collaboration. The selected studies will be screened based on relevance to developer workflows and empirical evaluation of AI programming tools. After collecting the relevant literature, the studies will be analyzed to extract common research methods, datasets, evaluation techniques, and reported findings. The results will then be synthesized to identify recurring themes, patterns, and open challenges related to how developers interact with AI assistants during software development.

## 4. Measuring Success

Success in this project will be demonstrated through a systematic synthesis of the selected literature and the identification of consistent themes across empirical studies. After collecting relevant papers, the findings will be analyzed by categorizing studies based on their research focus, methodologies, and reported outcomes related to AI-assisted programming. The analysis will compare how different studies evaluate developer productivity, workflow changes, and interaction patterns with AI tools. By organizing results into common dimensions such as task types, developer behaviors, and reported benefits or limitations, the project will produce a structured overview of how generative AI influences programming practices. The effectiveness of the approach will be reflected in the ability to clearly summarize the current state of research, highlight converging evidence across studies, and identify gaps and open research questions that remain insufficiently explored.

## References


[1] C. Bird et al. Taking flight with copilot: Early insights and opportunities of ai-powered pair-programming tools. In *Proceedings of the IEEE/ACM International Conference on Software Engineering*, 2023.

[2] R. P. L. Buse and W. Weimer. Learning a metric for code readability. *IEEE Transactions on Software Engineering*, 36 (4):546–558, 2010.

[3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer. The impact of ai-powered code completion on developer productivity. *arXiv preprint arXiv:2302.06590*, 2023.

[5] M. P. Robillard, R. J. Walker, and T. Zimmermann. Recommender systems for software engineering. *IEEE Software*, 27 (4):80–86, 2014.